Lecture 11: The Cook-Levin Theorem

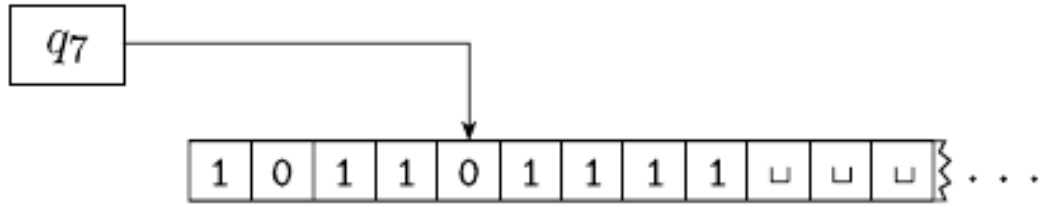
Theory of Computation

Ziyi Guan

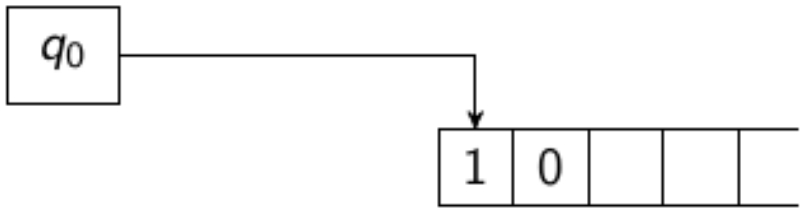
EPFL School of Computer and Communication Sciences

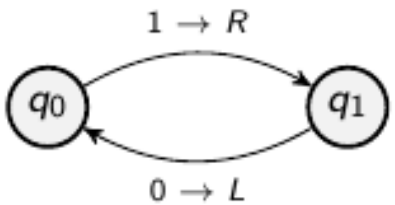
Review: Turing Machines

Turing Machine



- Infinite tape
- Tape alphabet contains input alphabet plus ⊔ (blank symbol) plus maybe more symbols
- Head has states (corresponding to the finite control automata)
- Exactly **one** Accept state and exactly **one** Reject state *(where computation immediately ends)*
- Remaining states *"computation in progress"*
- May never reach an accept state. May never halt!





Formal Definition of a TM

A **Turing Machine** is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{accept} \in Q$ is the accept state, and

$\{w : w \text{ has an equal number of 0s and 1s}\}$

Easy (efficient) to write an algorithm to count number of 1s and 0's — let's try to implement on a TM

(In this Part II of the course, we do not care about running time...)

$\{w : w$ has an equal number of 0s and 1s$\}$

Easy (efficient) to write an algorithm to count number of 1s and 0's — let's try to implement on a TM

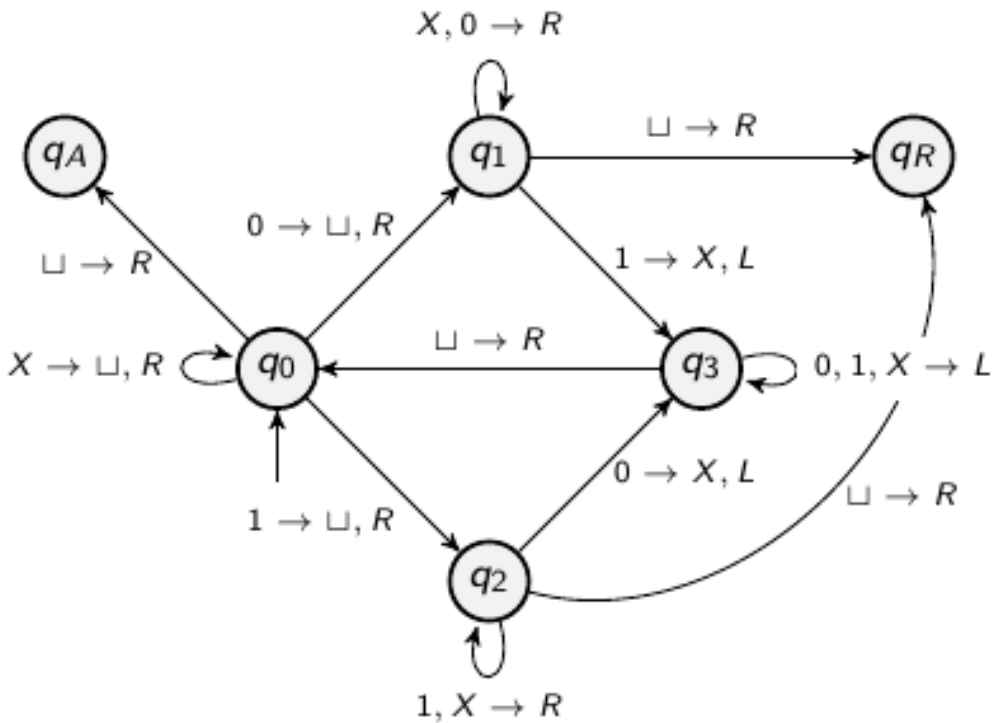
(In this Part II of the course, we do not care about running time. . .)

Our approach: check for each 0 (or 1) there is a corresponding 1 (or 0)

- Scan the input from left to right
- Whenever we encounter an uncrossed 0 or 1, we "remove" it and proceed right to find a corresponding 1 or 0 that we cross
- We keep doing this (2) until either we cross off all the letters (accept) or we fail to find a pair for one of the letters (reject)

$\{w : w$ has an equal number of 0s and 1s$\}$

$Q = \{q_0, q_1, q_2, q_3, q_A, q_R\}$
$\Sigma = \{0, 1\}$
$\Gamma = \{0, 1, \sqcup, X\}$ – X crossed off letter



Configurations of a TM

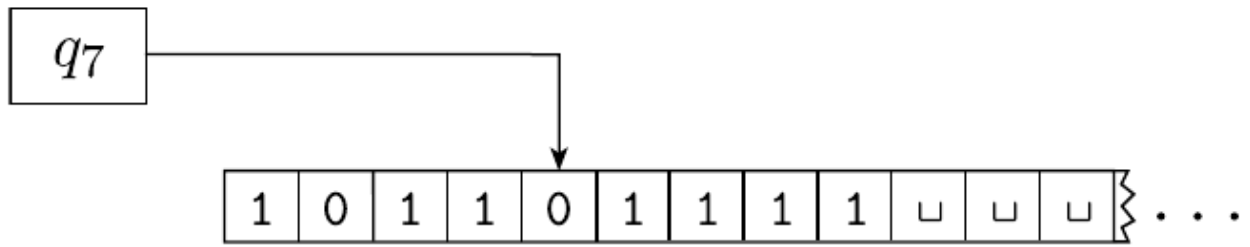
As a Turing machine computes, changes occur in the current state, the current tape contents, and the current head location. A setting of these three items is called a *configuration* of the Turing machine.

Representation: We write uqv where $u, v \in \Gamma^*$ and $q \in Q$ for the configuration where

- current state is q
- current tape content is uv
- the current head location is the first symbol of v

(Cells whose contents are unspecified are blank. If $u = \varepsilon$ then Head at leftmost cell.)

Example: A TM with configuration $1011q_701111$



Transitions: Given configuration uaq_ibv where $a, b \in \Gamma$, $u, v \in \Gamma^*$ and state $q_i \in Q$, we move to

- uq_jacv if $\delta(q_i, b) = (q_j, c, L)$
- $uacq_jv$ if $\delta(q_i, b) = (q_j, c, R)$

Computation:

- Starting configuration is $C_1 = q_0w$ on input $w \in \Sigma^*$
- Obtain new configurations $C_2, C_3, \ldots$ by valid moves/transitions
- Accept and halt if a configuration with the state q_{accept} is reached
- Reject and halt if a configuration with the state q_{reject} is reached

What if the computation doesn't halt (i.e., loops)?
Does it mean some configuration is repeated?

Previously:

- NP-completeness
- $SAT \leq_P 3SAT$

Today:

- Cook-Levin Theorem: SAT is NP-complete

Quick Review

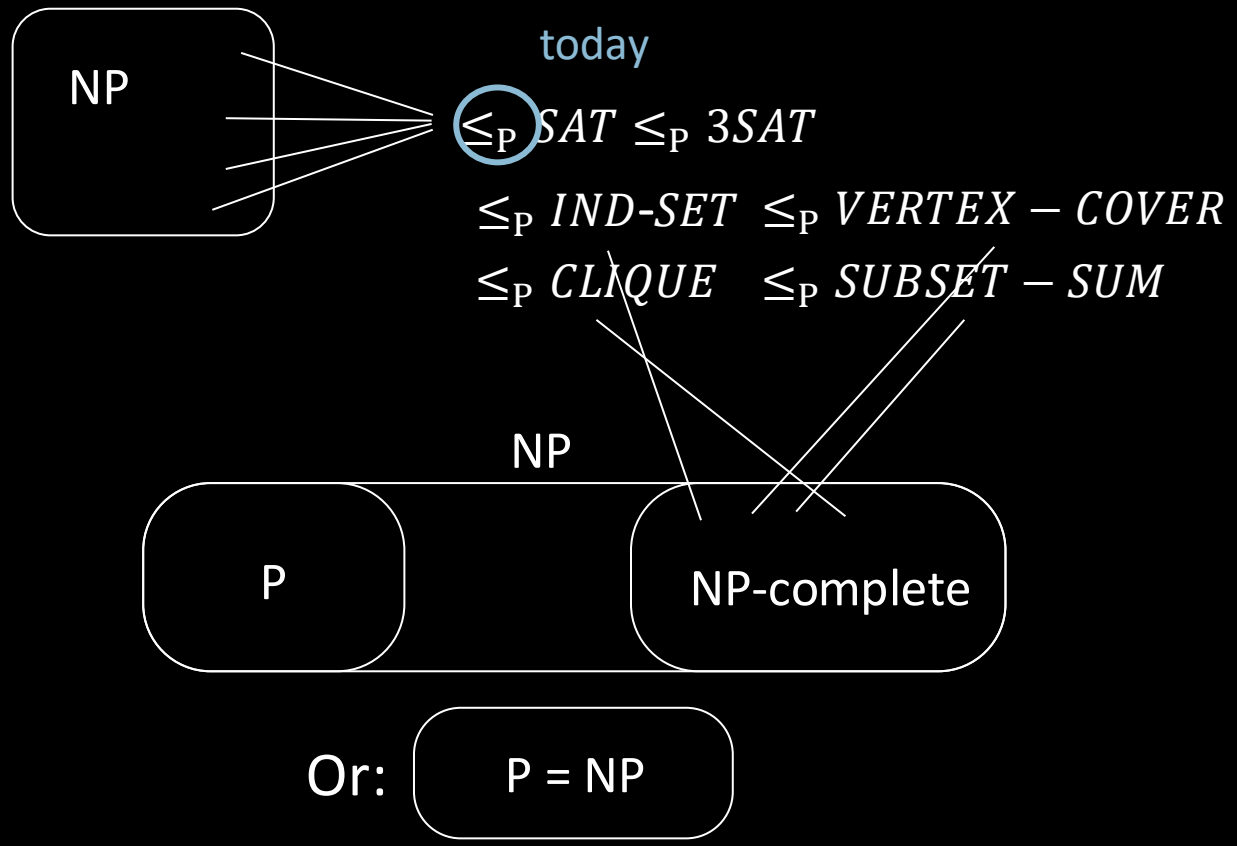
Defn: B is NP-complete if
1) $B \in$ NP
2) For all $A \in$ NP, $A \leq_P B$

If B is NP-complete and $B \in$ P then P = NP.

Importance of NP-completeness
1) Evidence of computational intractability.
2) Gives a good candidate for proving P ≠ NP.

To show some language C is NP-complete, show $SAT \leq_P C$.

or some other previously shown NP-complete language



Cook-Levin Theorem (idea)

Theorem: SAT is NP-complete

Proof: 1) $SAT \in NP$ (done)

2) Show that for each $A \in NP$ we have $A \leq_P SAT$:

Let $A \in NP$ be decided by NTM M in time n^k.

Give a polynomial-time reduction f mapping A to SAT.

f: $\Sigma^* \rightarrow$ formulas

$f(w) = \langle \phi_{M,w} \rangle$

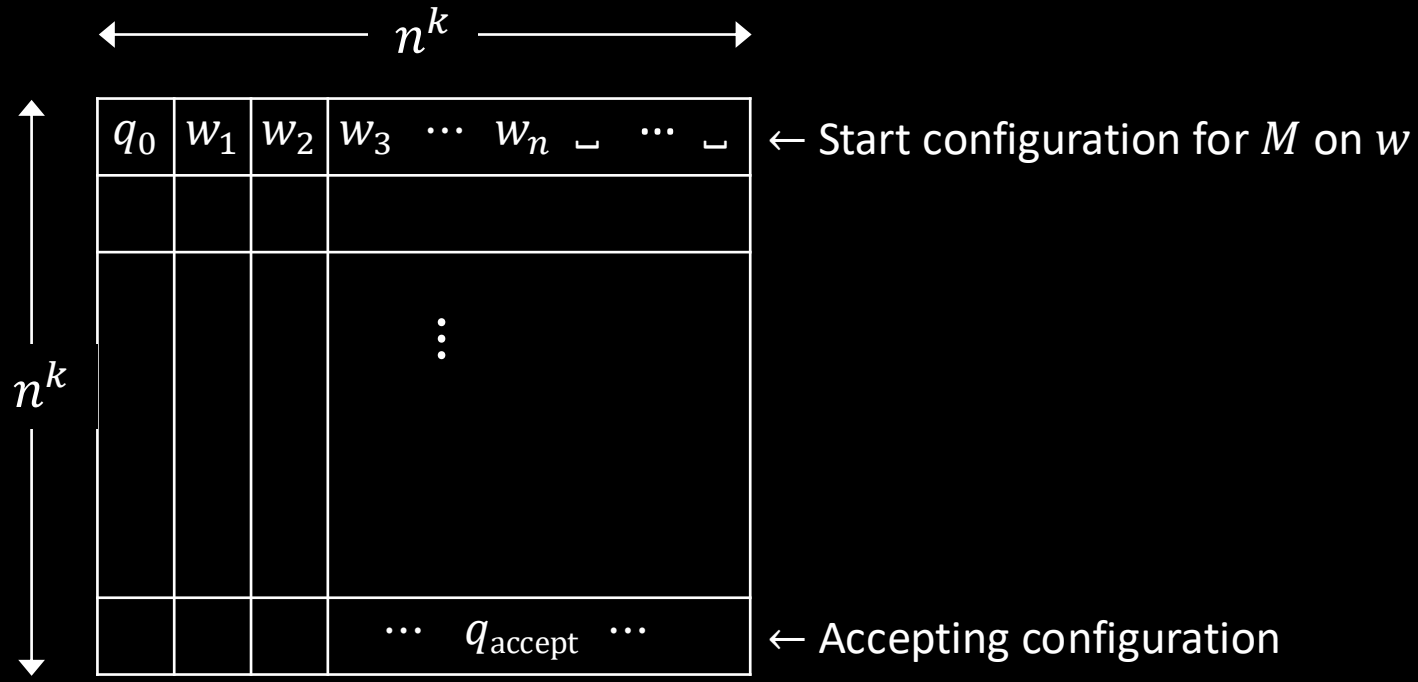
$w \in A$ iff $\phi_{M,w}$ is satisfiable

Idea: $\phi_{M,w}$ simulates M on w. Design $\phi_{M,w}$ to "say" M accepts w.

Satisfying assignment to $\phi_{M,w}$ is a computation history for M on w.

Tableau for M on w

Defn: An (accepting) tableau for NTM M on w is an $n^k \times n^k$ table representing an computation history for M on w on an accepting branch of the nondeterministic computation.

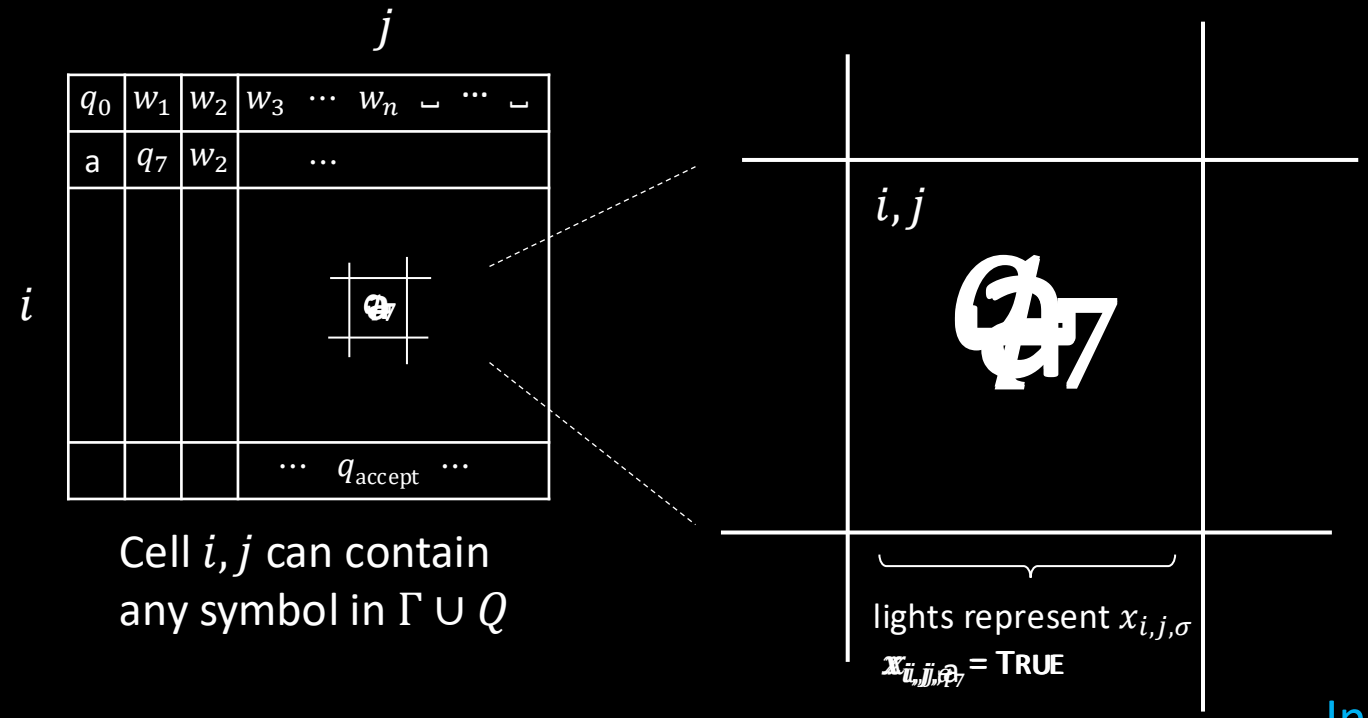


Construct $\phi_{M,w}$ to "say" M accepts w.

$\phi_{M,w}$ "says" a tableau for M on w exists.

$\phi_{M,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$

Constructing $\phi_{M,w}$: ϕ_{cell}



Cell i, j can contain any symbol in $\Gamma \cup Q$

The variables of $\phi_{M,w}$ are $x_{i,j,\sigma}$ for $1 \le i, j \le n^k$ and $\sigma \in \Gamma \cup Q$.

$x_{i,j,\sigma}$ = TRUE means cell i, j contains σ.

$\phi_{M,w}$ "says" a tableau for M on w exists.

$\phi_{M,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$

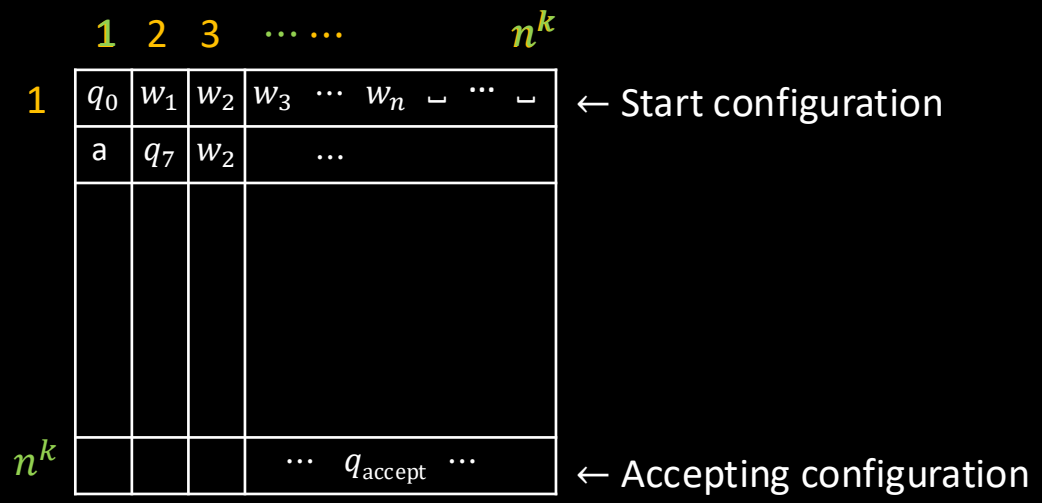
ϕ_{cell} "says" exactly one light is on per cell i.e., exactly one $x_{i,j,\sigma}$ = TRUE for each i, j .

In every cell at least one light and at most one light

$$\underbrace{x_{i,j,\sigma_1} \vee x_{i,j,\sigma_2} \vee \cdots \vee x_{i,j,\sigma_t}}$$

$$\phi_{\text{cell}} = \bigwedge_{1 \le i,j \le n^k} \left(\bigvee_{\sigma \in \Gamma \cup Q} x_{i,j,\sigma} \wedge \bigwedge_{\substack{\sigma,\tau \in \Gamma \cup Q \\ \sigma \neq \tau}} \left(\overline{x_{i,j,\sigma} \wedge x_{i,j,\tau}} \right) \right)$$

Constructing $\phi_{M,w}$: ϕ_{start} and ϕ_{accept}



$\phi_{M,w}$ "says" a tableau for M on w exists.

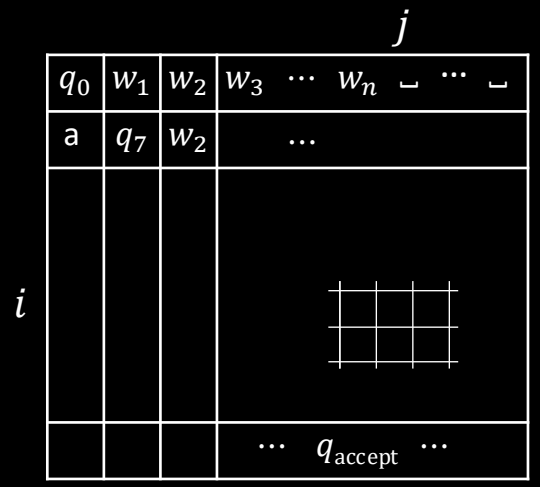
$\phi_{M,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$

ϕ_{cell} done ✓

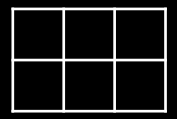
$\phi_{\text{start}} =$

$$\phi_{\text{accept}} = \bigvee_{1 \le j \le n^k} x_{n^k, j, q_{\text{accept}}}$$

Constructing $\phi_{M,w}$: ϕ_{move}



2 × 3 neighborhood



$\phi_{M,w}$ "says" a tableau for M on w exists.

$\phi_{M,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$

✓ ✓ ✓

Legal neighborhoods: consistent with M's transition function

potential examples:

a	q_7	b
q_3	a	c

a	b	c
a	b	c

a	b	c
a	b	q_5

a	b	c
d	b	c

Illegal neighborhoods: not consistent with M's transition function

examples:

a	b	c
a	d	c

a	b	c
a	q_2	c

a	q_7	c
a	b	c

a	q_7	c
q_3	d	q_4

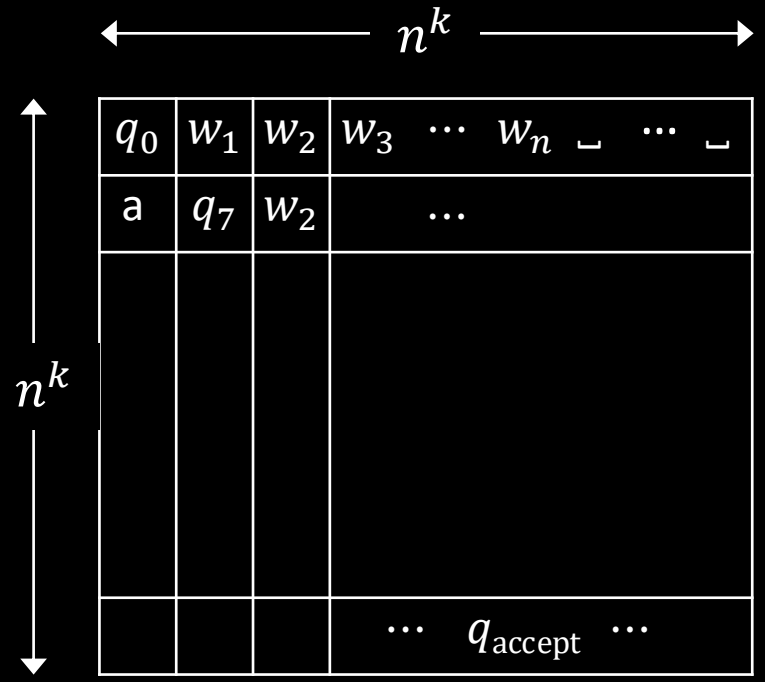
Claim: If every 2 × 3 neighborhood is legal then tableau corresponds to a computation history.

$$\phi_{\text{move}} = \bigwedge_{1<i,j<n^k} \left(\bigvee_{\text{Legal}} \left(x_{i,j-1,r} \wedge x_{i,j,s} \wedge x_{i,j+1,t} \wedge x_{i+1,j-1,v} \wedge x_{i+1,j,y} \wedge x_{i+1,j+1,z} \right) \right)$$

r	s	t
v	y	z

Says that the neighborhood at i, j is legal

Conclusion: SAT is NP-complete



Summary:

For $A \in$ NP, decided by NTM M,
we gave a reduction f from A to SAT:

f: $\Sigma^* \rightarrow$ formulas

$f(w) = \langle \phi_{M,w} \rangle$

$w \in A$ iff $\phi_{M,w}$ is satisfiable.

$\phi_{M,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$

The size of $\phi_{M,w}$ is roughly the size of the tableau for M on w, so size is $O(n^k \times n^k) = O(n^{2k})$.

Therefore f is computable in polynomial time.